Paso 7 — Retest OOS en SQX: qué se configura, con qué costes, qué se cosecha y qué puede fallar sin avisar

Fecha: 2026-10-08 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** el paso 7 del workflow (la tarea `OOS` de SQX que sigue al Build), la cosecha de sus dos databanks hacia Python y la regla dura 1. El paso 8 tiene su propio dossier (`puerta-is-oos-paso-08-especificacion-2026-10-06`): aquí sólo se dice qué le entrega el 7.

Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas.

Qué es: el documento de consulta del paso 7. Este paso **no juzga ni calcula estadística**: es una tarea de SQX que vuelve a probar, a otros costes y en otras fechas, las estrategias que el Build dejó en `Results`. Por eso la sección 2 es corta (conceptos) y la 4 no lista reglas de `criteria.yaml` —el paso no tiene ninguna—, sino las negativas y comprobaciones que hay en el código.

De dónde salen los números. - *Costes, fechas, motor y precisión:* leídos del `project.cfx` archivado de `Trade_NIKKEI225_M30_volatilidadL` (NIKKEI225, M30, largas), copia del 2026-10-04 en `AlgoData/projects/snapshots/Trade_NIKKEI225_M30_volatilidadL/20261004-210011/`, miembros `Build-Task3.xml` (título `CONSTRUCCION`) y `Retest-Task1.xml` (título `OOS`). Los comparé con `assets/symbols/NIKKEI225.yaml`: coinciden en todas las cifras. La copia es de las 21:00 y la cosecha de las 15:59, es decir, 5 h posterior (calculado por el revisor, por la hora de los ficheros); que los costes coincidan con `assets/` prueba que el cfx está bien escrito, no que fuera el de la corrida: no hay log del 6+7 de ese proyecto (el resumen del autopiloto `20261004-154759` empieza ya en el paso 8) y la tarea `OOS` de la copia trae `active="false"` en `config.xml:4`. - *Cosecha:* `AlgoData/harvest/Trade_NIKKEI225_M30_volatilidadL/Results/2026-10-04/` (10 pares, 3.053 operaciones) y `AlgoData/raw/Trade_NIKKEI225_M30_volatilidadL/{Results,OOS}/2026-10-04/`. Para ver una población entera usé `AlgoData/harvest/Test_USDJPY_M30_doubleROCLong_10k_B5/Results/2026-10-08/` (10.000 pares, USDJPY M30). Esas cifras las **recalculé yo** desde los parquet con un script ligero (`scratchpad/paso07/check_b5.py`, una sola pasada, sin muestreo ni simulación) y lo digo cada vez. - *Código:* el árbol de trabajo de hoy, no el de la corrida (la corrida es del 04-10; el código de la cosecha dice `d8e1ffe-dirty`, y su `manifest.json` declara `join: identity (sha256…), file name as fallback`: la cosecha se hizo con una versión que unía por identidad con el nombre de respaldo, mientras que el código de hoy empareja sólo por nombre).

El commit que describo: `b2b4c8c`. **Ficheros del paso con cambios sin confirmar** (`git status --short`): `assets/_build.yaml`, `assets/_policy.yaml`, `sqx/projects/setups.py`, `sqx/projects/stages.yaml`, `sqx/projects/workflow.py`, `pipeline/autopilot/{run,plan,judge,facts,blind,tailrun,variants}.py`, `studies/screening/gate/{inputs,report,level3ab,level3c}.py`, `ui/daemon/launch/{run,liverun,configure}.py`, `core/tradepack.py`, `sqx/export/{export_trades,archive_logs}.py`, `knowhow/databanks/{INDEX,memory-vs-disk-exporter}.md` (y `autosync-nothing-on-disk.md` borrada), `knowhow/export/{INDEX,archived-trades-vs-`

`reclocked-bars}.md`. **Importa uno:** `sqx/projects/setups.py` con `OOS_RANGE` (líneas 139 y 172-175) está **sin confirmar**; el commit `b2b4c8c` no reescribe el rango fuera de muestra de la tarea. El cfx del 04-10 ya lo trae reescrito, así que la corrida es del código de hoy, no del confirmado.

Ninguna línea de este documento está congelada.

---

0. En una página

La pregunta del paso. Las estrategias que el Build dejó en `Results`, ¿cómo se comportan en las fechas que el generador no vio, pagando los costes de esas fechas? El paso 7 no contesta: produce el segundo juego de números (databank `OOS`) con el que el paso 8 contestará.

Lo cargado hoy: ninguna regla en este paso. Lo único parecido a una comprobación son las negativas del arranque (sección 4) y la regla R1a del paso 8 («la estrategia del build aparece en el retest», `criteria.yaml:74`), que decide qué pasa si falta una.

Lo que sale en la corrida real (Nikkei, M30): la tarea `OOS` lee `Results`, vuelve a probar cada estrategia del 2020.01.01 al 2023.12.31 con spread 474,91 puntos y deslizamiento 166,22 puntos (el Build: 304,11 y 106,44) y escribe `OOS`. Misma pareja de bases de datos, mismo motor, misma precisión, misma gestión de dinero, misma sesión. En seis cosechas de otros proyectos el `OOS` sale con todas las estrategias del build y 0 sin gemela: cuatro de 10.000 (USDJPY 10k B2 a B5), una de 3.000 (USA500 H1 del 05-10) y una de 200 (USDJPY donchian M30) (recontadas de sus `manifest.json`; calculado por el revisor). En otras con menos `OOS` que build (p. ej. USDJPY doubleROCLong 10k del 07-10: build 10.000, `OOS` 56) no se ha comprobado si es un corte posterior del paso 8 o de SQX; la tarea `OOS` lleva además un `StopCondition` de 10 minutos (`Retest-Task1.xml:216`) del que no se ha comprobado si SQX lo aplica a un retest.

Las diez cosas que conviene saber antes de leer el resto:

1. **Lo que cambia entre el Build y la tarea OOS y afecta al número es: las fechas, el spread/deslizamiento y el rango fuera de muestra.** Lo demás que importa para que las dos cifras se puedan comparar (motor `MetaTrader5 (hedged)`, precisión 1, timeframe, sesión, tamaño por ATR activo, comisión 0 %, swap) es idéntico en el cfx. Otras diferencias del cfx son inertes (casillas apagadas) o filtros del Build (apartado 3.2).
2. **El spread del OOS es 1,56 veces el del Build en puntos, pero el mismo porcentaje del precio.** 304,11 puntos × 0,01 / 17.525,7 = 0,017352 %; 474,91 × 0,01 / 27.368,7 = 0,017352 %. El dueño lo decidió así el 2026-10-04. La ficha `costs/funded-tick-spread-vs-price` mide en los índices que el spread en puntos se mueve poco con el precio (en NIKKEI225 la elasticidad es -0,64: no es plano del todo; sí lo son USATEC, USA500 y DJ30) y da para FTMO JP225.cash 487-1.094 pt según el año. Con esa lectura **tanto el Build (304) como el OOS (475) pagan menos puntos de los medidos**, el OOS por poco y el Build por bastante: el % del dueño reparte la infraestimación de forma desigual y deja el Build más barato que el OOS en proporción 1,56 (apartado 5, punto 1).
3. **El swap es el de hoy en las dos tareas** (-4,78 % anual largo, -1,87 % corto), también para unas fechas de 2020-2023 con otros tipos de interés. Como es un % anual del nocional, el mismo % cobra en 2020-23 en torno al 40 % de los puntos de hoy (precio 27,4k frente a 68k; calculado por el revisor, apartado 3.2).

4. **La métrica de OOS que SQX deja tiene sólo 13 columnas y le faltan el nº de operaciones, el drawdown y el Max DD %** (`core/sqxview.py:24-29`). El nº de operaciones del OOS sale de las operaciones exportadas, no de la métrica.
5. **La cosecha lee los `.sqx` del disco, no la memoria de SQX.** Por eso la regla 1 (cada sync borra lo que no está en memoria) llega al paso 7: lo que se pierde en el sync de parada no aparece en la cosecha, y el compare de arranque **no vigila `Results` ni `OOS` en una corrida 6+7** porque son salidas de la propia corrida (apartado 3.5). Una pérdida en `OOS` acaba en la regla R1a; una pérdida en `Results` no la ve nadie: la cosecha cuenta lo que hay en disco y la población que llega al paso 8 sale más pequeña sin que ningún fichero lo diga.
6. **El emparejamiento IS-OOS es por nombre de fichero** (`pairing.py:23-31` , decisión del 2026-09-26) y la ficha del 2026-10-08 `sqx-format/names-reused-between-builds` dice que **nunca** hay que emparejar por nombre entre builds. Si `OOS` se queda con estrategias de un build anterior (un reintento no lo vacía), el emparejamiento puede unir dos estrategias distintas sin error. Hoy sólo avisa un contador de identidades distintas, que se imprime y no se guarda en `manifest.json` (apartado 5, punto 3).
7. **En la cosecha de la corrida de ejemplo, 9.990 de 10.000 estrategias «no están en el OOS».** No las mató SQX: son el recorte posterior del paso 8 (modo desarrollo, sorteo de 10) sobre el databank `OOS` . El fichero `missing_oos.csv` mezcla las dos causas y no las distingue (apartado 3.6).
8. **La ficha `databanks/databank-metrics-is-oos` está desfasada:** describe la exportación de métricas por una vista `.vw` con SQX arrancado; el código hoy las lee de los `.sqx` sin arrancar nada (`export_metrics.py:22-29`).
9. **El IS parece empezar tarde; el OOS no.** En el IS, las 10 primeras operaciones de la corrida llegan 148-184 días después del 2011-09-19 (una a 627; cociente mediano frente a la brecha media entre operaciones, 9,0); en el OOS el mismo cociente es 1,06, es decir, el tiempo esperado hasta la primera señal (calculado por el revisor, desde las operaciones de la cosecha). Parece un arranque comido en el IS (feed o calentamiento); no se ha mirado `NIKKEI225_M1` desde 2011-09-19 (apartado 5, punto 11).
10. **Los 10 códigos de `AutomaticDismissal` ya están identificados** (apartado 5, punto 6). Cuatro de ellos (operaciones con P/L cero, de duración cero, operación atípica, operaciones ambiguas) tocan estrategias con pocas o raras operaciones y pueden dispararse justo en un OOS de 4 años; y no es la única vía por la que SQX deja una estrategia fuera del `OOS` .

---

1. Qué es el paso y cómo «decide»

1.1 Dónde está

Es el paso 7 de `WORKFLOW.md` («Retest OOS en SQX — tarea propia, costes de `oos1` », fila 32). Va justo detrás del 6 (el Build) y, en el autopiloto, **comparten una sola arrancada de SQX**: `pipeline/autopilot/run.py:83-92` (`merged`) junta pasos SQX consecutivos y los lanza como `6+7` . SQX encadena las dos tareas él mismo (el OOS empieza 0,19 s después del Build y parar y arrancar entre medias costaría unos 95 s, según la docstring de `pipeline/autopilot/run.py:62-63` ; esas dos cifras no están en la ficha `knowhow/sqx-drive/live-chain-without-restart` , que da 63 s el Build, 2,3 s el OOS, 28 s de inicio y 30 s de parada).

Lo que el paso lleva a cabo está en tres sitios:

qué	dónde
qué tarea es el «paso 7»	`sqx/projects/stages.yaml:12` → `oos: [OOS]`; `ui/daemon/workflow/steps.py:29` y `:80` (`"7": "oos"`)
con qué costes y fechas se escribe	`sqx/projects/configure.py:26-27,90-100,131` (`BY_TASK = {"Build": "build"}`, resto `oos1`) y `sqx/projects/setups.py:76-175`
con qué motor y precisión	`sqx/projects/doctrine.py:37-40` + `assets/_build.yaml:72` (motor) y `:98-100` (precisión)

1.2 Cómo «decide»

No decide. No hay umbral ni veredicto. Lo que hay son tres cosas:

- **Una tarea de SQX sin filtros.** Ninguna condición de aceptación de la tarea `OOS` está encendida (el cfx no tiene un solo `use="true"` dentro de `<Rankings>`, líneas 138-220; las que enseña, p. ej. el Sharpe de la línea 149, están en `use="false"`); el builder las apaga en toda tarea `Retest` de un workflow (`builder.py:99-101`, `silence`). El motivo está en `assets/_build.yaml:125-135`: con una condición viva SQX no escribe la estrategia que no la cumple, y Python pierde las muertas, que son la mitad de la evidencia.
- **Una exportación al parar.** Cada `bin/sqx-worker.sh stop` ejecuta `ui/daemon/loader/afterrun.py:111-135`, que exporta métricas, operaciones y la cosecha de cada databank del proyecto que se ha ejecutado.
- **Una cosecha** (`studies/screening/gate/harvest.py`) que junta `Results` y `OOS` y es lo que lee el paso 8.

---

2. Conceptos que hacen falta (sin estadística)

Retestear. Una tarea de tipo *Retest* de SQX toma cada estrategia de su databank de entrada, **no toca su lógica ni sus parámetros**, la vuelve a ejecutar con la configuración de la propia tarea (fechas, spread, deslizamiento, comisión, swap, motor, precisión) y escribe el resultado en su databank de salida. La ficha `sqx-format/retest-rewrites-sqstats` lo mide: las estadísticas guardadas dentro del `.sqx` se reescriben; hay que comparar con la estrategia **retesteada**, nunca con la cifra heredada.

Por qué es una tarea aparte y no «el OOS del Build». Un `<Setup>` de SQX lleva **un** spread y **un** deslizamiento (ficha `sqx-format/oos-range-marker`, `per-task-costs`). Si el Build probase 2011-2023 de una vez, un tramo de doce años tendría un solo coste. Con tareas separadas, cada una escribe su propio databank y paga sus costes. Las dos cosas que consigue: (a) cada tramo paga lo que le toca; (b) el Build **no ve** el OOS (sólo carga barras hasta el 2019.12.31, apartado 3.1).

Dos bloques de muestra. Cada `.sqx` guarda hasta dos bloques de estadísticas: el de muestra 10 («IS») y el de muestra 20 («OOS»). Una tarea sin rango marcado llena el 10; una con `<OutOfSample><Range/>` marcado llena el 20. SQX deja el otro a cero sin avisar.

Identidad de una estrategia. `core.sqxfile.identity` = SHA-256 del `strategy_Portfolio.xml` interior, sin las marcas que SQX cambia al retestear (`makeExternal`, `autoGenerated`). Tras esa limpieza, la identidad **no cambia** de `Results` a `OOS` (ficha `identity-differs-across-databanks`, 2026-10-06).

Reloj de una feed. SQX sella las barras y las operaciones en la zona horaria de su feed, nunca en UTC. `NIKKEI225_M1` es `EETUS` (Nueva York + 7 h, sin nombre IANA; ficha `export/feed-clock-timezones`).

---

3. Lo que hace el paso, pieza a pieza

3.1 Fechas: qué ve y qué carga cada tarea

Cifras del cfx archivado (NIKKEI225, M30, feed `NIKKEI225_M1`):

	Build (`CONSTRUCCION`)	**OOS (`OOS`)**	**de dónde**
ventana que opera (`<Setup dateFrom/dateTo>`)	2011.09.19 → 2019.12.31	2020.01.01 → 2023.12.31	`Build-Task3.xml` / `Retest-Task1.xml` (línea 117)
barras que carga (`<Symbol dateFrom/dateTo>`, ms)	1316390400000 → 1577836800000 = 2011-09-19 → 2020-01-01 00:00	1577836800000 → 1704067200000 = 2020-01-01 → 2024-01-01	ídem (`Symbol`, línea 753)
`<OutOfSample>`	sin rango, `showGraph="true"`	`Range 2020.01.01 → 2023.12.31`, `showGraph="false"`	línea 134-136

Las ventanas salen de `assets/_policy.yaml:102-106` (`build`, `oos1`) y las pinta `setups.bounds` (`setups.py:15-28`): `from: 2020, to: 2023` se vuelve 2020.01.01 y 2023.12.31 (inclusivos). No hay hueco entre `build` y `oos1`.

Consecuencia que he comprobado en los datos. La tarea OOS carga barras **desde el 2020-01-01** (`Symbol dateFrom`, `Retest-Task1.xml:753`) y el Build termina en esa misma fecha: ninguna barra anterior al 2020-01-01 está cargada en el OOS, así que no hay contaminación desde el Build. Si hay calentamiento de indicadores, le quita operaciones al OOS, no se las da (`ReservedBars 50`). En la corrida real la primera operación de las diez estrategias cae entre el 2020-01-08 y el 2020-02-05 (7 a 35 días). *Medición del revisor:* los días hasta la primera operación tras el 2020-01-01 son 7,2 · 7,8 · 8,3 · 9,4 · 12,9 · 16,1 · 16,1 · 21,2 · 26,6 · 35,5; la brecha media entre operaciones de cada estrategia (1.460 días / nº de operaciones) va de 7,8 a 25,2 días; el cociente mediano entre ambas es 1,06 (calculado por el revisor). Si el calentamiento se comiera semanas el cociente sería muy superior a 1: no hay evidencia de que se coma el principio, y el retraso es el tiempo esperado hasta la primera señal. En el IS el mismo cálculo da otra cosa (sección 5, punto 11).

El rango `<OutOfSample><Range>` importa: la ficha `oos-range-marker` documenta que el donante lleva el `oos1` de XAUUSD (2018-2022) y que en un activo con otros años (índices 2020-2023) el retest se parte en dos bloques en la fecha equivocada. `setups.py:139,172-175` lo reescribe al `oos1` del activo (**sin confirmar**, ver cabecera). En la corrida real el rango y la ventana coinciden.

3.2 Costes: cifra a cifra, Build contra OOS

Del cfx archivado, `<Setup>` de cada tarea (`Build-Task3.xml` líneas 198-214 — `Chart spread` en la 199, `Swap` en la 212— y `Retest-Task1.xml` líneas 117-132):

coste	Build	OOS	cómo se escribe	de dónde sale
spread (`<Chart spread>` , puntos)	**304,11**	**474,91**	`setups.py:93`	`assets/symbols/NIKKEI225.yaml` : `spread_is.use` / `spread_oos.use`
deslizamiento (`slippage=` , puntos)	**106,44**	**166,22**	`setups.py:91-92`	`slippage_is.use` / `slippage_oos.use`
comisión	`PercentageBased` 0,0 (la de tamaño, `SizeBased 8` , **apagada**)	idéntica	`setups.py:95-99`	`costs.commission.use.build` / `.oos1` : 0,0 % (dueño, 2026-09-29: cero en índices)
swap	`percent` , largo **-4,78**, corto **-1,87**, triple el viernes, rollover 23:00	idéntico	`setups.py:101-105`	`swap_long.use` / `swap_short.use` (iguales en todos los tramos)
`minDist`	10	10	donante	—

Qué significan las unidades y cómo lo comprobé:

- **Puntos.** El instrumento tiene `tickSize 0.01` (`Retest-Task1.xml:754`); 1 punto = 0,01 de precio. Por eso 474,91 puntos = 4,7491 puntos de índice. Lo comprobé con el porcentaje de la ficha del activo: 0,017352 % del precio medio del tramo, en los dos (cuenta de la cabecera, punto 2 de la sección 0). El fichero lo redondea a «0,0174 %».
- **Deslizamiento = 35 % del spread** del mismo tramo (dueño, 2026-10-04): 166,22 / 474,91 = 0,35000 y 106,44 / 304,11 = 0,35000.
- **Swap en %.** `swap_long.use` está anotado como «% ANUAL (SQX divide /100 y /360)». Es la nota del fichero. La fórmula de SQX para `percent` es `tamaño × valor del punto × precio de apertura × (swap / 100 / 360)` (`knowhow/costs/swap-types.md:18` , `assets/_classes.yaml:97`), así que las unidades (% anual del nocional, entre 360) son las correctas. Comprobación del revisor con la cifra del yaml (calculado por el revisor): el swap largo de FTMO, -906,33 pt por noche, son 9,0633 puntos de índice; 9,0633 × 360 / 0,0478 = 68.259; el corto de Hantec, -355,65 pt, 3,5565 × 360 / 0,0187 = 68.467; el precio guardado en el yaml es 67.087,52 (2026-10-01): coinciden a ±2 %. Matiz: al ser % del nocional, el «swap de hoy» no cobra en 2020-23 lo que cobra hoy. Una noche larga a 92 lotes (calculado por el revisor): Build (precio medio 17.525,7) 2,33 puntos de índice = 13,5 USD; OOS (27.368,7) 3,63 puntos = 21,1 USD; hoy (68.258) 9,06 puntos = 52,6 USD. El mismo % en 2020-23 cobra, pues, en torno al 40 % de los puntos de hoy.
- **Ilustración en dólares** (aproximada: no sé cómo reparte SQX el spread y el deslizamiento entre la entrada y la salida; ficha `fill-and-pricing`). Valor del punto de precio por lote: `pointValue 0.063137` (`Retest-Task1.xml:754`). Una operación de 100 lotes: el spread del Build vale 3,0411 × 100 × 0,063137 = **19,20 USD**; el del OOS, 4,7491 × 100 × 0,063137 = **29,98 USD**. El deslizamiento: 6,72 USD y 10,49 USD. Con el tamaño real de la corrida (media 92 lotes, de 16 a 325 en las 3.053 operaciones) el orden es el mismo. *Ilustrativo, no medido.*

Lo que no cambia entre las dos tareas (comparado en el cfx; lo importante es que un OOS y un IS con distinta configuración no son comparables). El método de dinero activo `ATRRiskBasedSizingFixedRisk` (1000, 20, 4, 2, 0,01, 500), `InitialCapital 100000` y `maxDrawdown 30` son idénticos; los métodos de dinero **apagados** sí difieren (ver debajo):

ajuste	valor en las dos tareas
motor (`engine`)	`MetaTrader5 (hedged)` (`_build.yaml:72` , `doctrine.py:40`)
precisión de backtest (`testPrecision`)	**1** = sólo el timeframe elegido, M30, sin bajar a M1 (`_build.yaml:98-100` : `build: 1` , `default: 1` ; dueño, 2026-10-02: «nada de retest a M1»)
timeframe y feed	`NIKKEI225_M1` en M30
gestión de dinero	`ATRRiskBasedSizingFixedRisk` : 1.000 USD, ATR de 20 periodos × 4, 2 decimales, mínimo 0,01 y máximo 500 lotes; capital inicial 100.000; drawdown máximo 30
sesión	`JP225.cash_ftmo`
opciones de trading	`ExitOnFriday` sí (21:00 = 75600 s), `RealisticGapsHandling` sí, `ReservedBars` 50, `No Session`

Cuatro parámetros de `BuildTradingOptions` **sí** difieren (`EODExitTime` , `SignalTimeRangeFrom` , `SignalTimeRangeTo` , `ExitAtEndOfRange`), pero son **inertes**: `ExitAtEndOfDay` y `LimitTimeRange` están en `false` en las dos. Lo anoto porque un cambio de esas dos casillas los activaría de forma desigual.

Otras diferencias que el doctrine no unifica, comparadas por el revisor entre `Build-Task3.xml` y `Retest-Task1.xml` y que no afectan al número del OOS: `AutomaticDismissal warnings` (Build `true` , OOS `false` ; con los 10 códigos descartando en las dos, la casilla —«Add automatic checks warnings to strategy even when not used for dismissal», `autoDismissalPopup.html:39-41` — no cambia nada; sección 5.6); la comprobación `MonteCarloManipulation` (Build `use="true"` , línea 589; OOS `false` , línea 558; `_build.yaml:108`); `Rankings` con condiciones vivas en el Build (nº de operaciones) frente a ninguna en el OOS; `DismissTooSimilarStrategies=true` sólo en el Build (línea 226); y `<RiskMoneyManagement>` en los métodos apagados (`FixedAmount` con RiskedMoney 10000 / Decimals 2 / MaxLots 500 en el Build frente a 200 / 1 / 10; `RiskFixedPctOfAccount` con otro formato; el Build lleva además `SimpleMartingaleMM` , `CryptoSizeByPrice` y `ATRRiskBasedSizing` , que el OOS no).

Las tres demás comprobaciones de la tarea OOS están apagadas: `RetestOnAdditionalMarkets` , `RetestWithHigherPrecision` y `MonteCarloRetest` (`use="false"` , líneas 223, 358 y 395).

3.3 Qué databank lee y cuál escribe

`Retest-Task1.xml:747-748` : **entrada** `Results` , **salida** `OOS` . `Results` es la salida del Build (`Build-Task3.xml:29208`). El orden y las entradas de todas las tareas las recompone `databanks.chain_databanks` (`sqx/projects/databanks.py:38-71`): cada tarea lee lo que escribió la anterior, salvo la primera.

Todos los databanks del proyecto están en `Auto-sync every 1 hour` (`config.xml` , líneas 25-52). Eso importa para la sección 3.5.

La tarea tiene `retestSelected="false"` : lee **todo** `Results` , no una selección.

3.4 Cómo se ejecuta y cuánto tarda

`bin/workflow.sh` → `pipeline/autopilot/run.py` : la acción es `6+7` (`sqx_step` , `run.py:60-80`): un solo `preflight.check` , un solo `execute` (`liverun` en sesión GUI, `launcher` si no), un solo `stop` . El fin es «Project finished» en el log de SQX (regla dura 18); el resumen trae una línea con los dos databanks. Líneas reales de `resumen.md` :

- `6+7 | sqx | 1405 | CONSTRUCCION: 0 → 1000 · OOS: 0 → 0 · OOS: 0 → 1000` (`Test_NIKKEI225_M30_simpleL_auto/20261005-055944/resumen.md:5`): el OOS **no encadenó** y se repitió solo. En `Test_NIKKEI225_M30_simpleL_fin/20261005-160743` la misma corrida tardó 1298 s y salió `OOS: 0 → 1000` de una vez; y en `Test_NIKKEI225_M30_simpleL_val/20261005-013319` salió `OOS: 0 → 0 · OOS: 1000 → 2000` (un reintento que duplicó; respalda el punto 3 de la sección 5).
- `6+7 | sqx | 5922 | CONSTRUCCION: 0 → 10000 · OOS: 0 → 10000` (`Test_USDJPY_M30_doubleROCLong_10k_B2/20261007-193439`).

Si el Build para por su tope de minutos (`ui/daemon/advance/buildcap.py`: un proyecto parado no pasa a la tarea siguiente), el 7 no se ejecuta con el 6; `run.py:164-169` lo vuelve a lanzar solo. Una medición antigua (`knowhow/perf/autopilot-dead-time.md`, 2026-10-01, 571 estrategias): paso 7 = 27 s de preparación + 35 s de SQX + 106 s de parada y exportación = 168 s.

Antes de arrancar, `preflight.judge` (`ui/daemon/launch/preflight.py:85-136`) rechaza: un Build fuera del custodio; una entrada sin ninguna estrategia en disco (`n_in`), salvo que sea la salida de una tarea anterior del mismo arranque (`made`); y cualquier otro motivo de `advance.busy` (candado de otra sesión, GUI del maestro).

3.5 Regla dura 1 en este paso: qué se puede perder y qué lo vigila

Secuencia de una corrida 6+7: arranque → el Build llena `Results` **en memoria** → SQX encadena el OOS, que lee `Results` de memoria y llena `OOS` en memoria → parada: la sincronización de cierre escribe los dos a disco → `afterrun` exporta desde el disco. El sync deletes: el sync de un databank cargado borra del disco los `.sqx` que no tiene en memoria (`knowhow/databanks/sync-deletes-unloaded-files`).

Lo que lo vigila:

- Antes de arrancar, `ui/daemon/launch/run.py:41-53` copia el proyecto a `AlgoData/projects/snapshots/<P>/<sello>/` (sin `log/`). Después, `compare` (`run.py:55-79`) compara los recuentos y deja la copia si **bajó algún databank que la tarea no escribe**. `restore` (`run.py:82-110`) devuelve de la copia los ficheros que el log nombra con «Cannot process strategy 'X'» (la carrera de dos syncs).
- **Hueco que he encontrado:** `compare` descarta las salidas de la corrida (`ui/daemon/launch/run.py:69-70`: `if d not in outputs`) y `restore` recorre sólo `b not in outputs` (`run.py:104`). En una corrida `6+7` las salidas son `Results` y `OOS` (`outputs = {t["output"] for t in chosen}`, `run.py:189`, `liverun.py:133`). **Ninguna de las dos protecciones mira esos dos databanks.** Además, la ruta por defecto del autopiloto es `liverun` (`pipeline/autopilot/run.py:77`, `live.mine`), y ahí (`ui/daemon/launch/liverun.py`, alrededor de las líneas 130-136) `compare` se ejecuta tras `live.sync` y antes de `worker.stop`, y `restore` no se llama nunca (sólo en `run.py:189-191`, ruta `launcher`): en la ruta por defecto ni siquiera se mide el sync de parada (comprobado por el revisor). Una carrera de sync que borrase una estrategia de `Results` o de `OOS` no se restauraría ni se avisaría. Qué queda depende del databank: una estrategia que el sync borre de `OOS` la ve la regla R1a como fallo; una que borre de `Results` **no la ve nadie**, porque la cosecha construye el build con lo que hay en disco de `Results` (`harvest.py`, `collect.index`) y un `.sqx` borrado no entra en `missing`, ni en `level0.py`, ni en `missing_oos.csv`; la población que llega al paso 8 es más pequeña sin que ningún fichero lo diga. El único rastro es el recuento `CONSTRUCCION: 0 → N` del resumen. No he visto el caso en los datos; hablo del código, no de un incidente.

- La regla 1 prohíbe además mandar ningún comando de recuento (`-databank action=count`) entre el arranque y la recogida: `count` sincroniza **desde** ficheros y borra lo que sólo está en memoria (`knowhow/databanks/databank-verbs`).

3.6 Qué se exporta, con qué reloj y con qué redondeo

Dos caminos paralelos desde los mismos `.sqx` en disco:

a) Métricas (`sqx/export/export_metrics.py` , `core/sqxview.py`). Una fila por estrategia con las columnas de la vista «Export Data View», leídas del `SQStats` de cada `.sqx` **sin arrancar SQX**. El comentario de `sqxview.py:9-13` dice que se midió (2026-09-27, USDJPY_emaCross_H1, 100 + 32 estrategias) que cada columna coincide con la exportación de la vista dentro de 6e-8 relativo (float32), salvo `R Expectancy` , que la vista **redondea a dos decimales**. No lo he repetido.

Las columnas que llena cada bloque (`sqxview.py:14-30`): el bloque IS tiene **21** (Net profit, nº de operaciones, Profit factor, Sharpe, Sortino, R Expectancy, Winning %, Stability, Drawdown, Ret/DD, CAGR/Max DD %, Calmar, DoF Ratio, Max DD %, PSR, RSquared, SQN, TRL Ratio, Ulcer, ZScore, Param Count); el bloque OOS tiene **13** (Annual % Return, CAGR/Max DD %, Calmar, Net profit, PSR, Profit factor, Ret/DD, SQN, Sharpe, Sortino, Winning %, Ulcer, ZScore). **Al OOS le faltan** nº de operaciones, Drawdown, Max DD %, Stability, RSquared, TRL Ratio, R Expectancy, DoF Ratio y Param Count. En la cosecha real la tabla de métricas tiene 38 columnas y las del OOS son esas 13 (`metrics.parquet` , comprobado).

b) Operaciones (`exportdrv.trades` → `orderstocsv` , empaquetadas por `core/tradepack.py`). Columnas: `Type, Open time, Open price, Size, Close time, Close price, Profit/Loss, Balance, Sample type, Close type, MAE ($), MFE ($)` + `identity` y `sample` .

- **Reloj.** Naive, en la zona de la feed: EETUS para `NIKKEI225_M1` (cfx: `timezone="EETUS"`). Las horas de apertura de la corrida caen sólo en :00 y :30 (1.453 y 1.600 de 3.053), es decir, en la rejilla de barras M30. Las fechas de `Setup` son fechas planas en ese mismo reloj; el ancho de banda en ms de `<Symbol>` es medianoche UTC (no he comprobado si el borde del 2020-01-01 cambia algo por las 7 h de EETUS; la primera operación del OOS es del día 8, así que los datos no lo muestran).
- **Etiqueta de muestra.** En `Results` todas las operaciones llevan `IST` ; en `OOS` , todas `OOS1` (2.869.662 y 1.162 filas respectivamente en `raw/` , recontadas). La ficha `export/orderstocsv-schema` avisa de que el `Sample type` «vale lo que valga el último retest de la estrategia».
- **Redondeo.** Precios y P/L a 2 decimales (tick 0,01; el tamaño de ATR con `Decimals 2`). En la cosecha, `Net profit` de las métricas es igual a la suma de `Profit/Loss` de las operaciones: en las 10 estrategias de Nikkei, diferencia 0,00 en las 20 sumas; **recalculado por mí** en las 10.000 de USDJPY B5: diferencia máxima 0,0038 USD, y 0 de 20.000 sumas pasan de 0,05 (efecto de float32). No es lo mismo que la curva diaria: ficha `export/harvest-equity-vs-trades` (la curva y la suma de operaciones terminan en cifras distintas en la mitad de los pares; causa no investigada).
- **La curva diaria empieza antes de la ventana.** `equity.parquet` del OOS arranca el 2019-11-27 (35 días antes de la ventana del 2020-01-01; la curva de ceros va del 2019-11-27 al 2020-01-08, donde está la primera operación; calculado por el revisor) con ceros (ficha `sqx-format/leg-curve-warmup`). Cortar por fechas de la política, no por el primer punto.
- **Una operación `EndTest` .** En `Results` aparece 1 operación con `Close type = EndTest` (2019-12-31 22:30): SQX cierra a la fuerza lo que sigue abierto al final de la ventana. En el OOS de Nikkei hay 0, y en el de USDJPY B5 tampoco.

c) La cosecha (`studies/screening/gate/harvest.py`) que usa el paso 8:

1. `collect.index` (`collect.py:23-41`) calcula la identidad de cada `.sqx` de `Results` y de `OOS`.
2. `pairing.pair` (`pairing.py:23-31`) empareja por **nombre de fichero** (y `collect.index` indexa por identidad: dos `.sqx` de un mismo databank con la misma identidad quedan en un solo registro sin aviso, y `len(build)` ya cuenta uno menos); devuelve `pairs` , `alias` (pares cuya identidad cambió) y `missing` (estrategias del build sin gemela).
3. `collect.tables` (`collect.py:96-166`) copia todos los `.sqx` a una carpeta de trabajo con prefijo `IS__` / `OOS__` (SQX nombra una estrategia por su fichero), lee métricas y curvas de los ficheros mientras un `orderstocsv` saca las operaciones en un solo arranque de JVM.
4. `collect.measured` (`collect.py:44-73`) **lee de los datos** qué bloque llenó cada databank (10 o 20) y **se niega** (`SystemExit`) si llenaron los dos. En la corrida: `sample_blocks: IS=(IS), OOS=(OOS)` .
5. Escribe `metrics.parquet` , `trades.parquet` , `equity.parquet` , `missing_oos.csv` y `manifest.json` . `set_aside` (`harvest.py:52-71`) aparta a `<databank>/_aparte/<día>.<hora>/` una cosecha del mismo día de **otro conjunto** de estrategias; con el **mismo** conjunto, otra pasada del mismo día la sobrescribe y las cifras de la primera no se guardan (`harvest.py` , `harvest_dir(..., date.today())`).

Qué contiene la cosecha real de Nikkei (comprobado): 10 pares, 1.891 operaciones IS y 1.162 OOS. Operaciones OOS por estrategia, de 58 a 187 (en 4 años: de 14,5 a 46,8 al año); fechas de apertura del 2020-01-08 al 2023-12-29, la última cierre el 2023-12-29 18:00: dentro de la ventana.

estrategia	Net profit IS	nº op. IS	Net profit OOS	nº op. OOS	operaciones/año OOS
4.13.338	53.725,38	198	22.721,35	126	31,5
14.3.479	13.238,53	189	4.689,84	98	24,5
19.2.363	1.687,38	161	2.311,43	66	16,5
13.3.390	14.822,49	143	5.342,23	58	14,5
15.9.352	2.944,26	224	5.626,37	167	41,8
16.5.352	10.406,15	304	9.004,81	187	46,8
18.5.355	42.167,99	161	5.391,96	98	24,5
10.11.484	20.297,25	154	14.769,03	106	26,5
3.11.426	22.508,34	135	19.669,83	92	23,0
3.9.284	32.989,54	222	10.197,30	164	41,0

(IS dura 8,3 años y OOS 4; las cifras de dinero no son comparables entre sí. Las operaciones por año las he calculado como nº/4; la regla de 35 al año es del paso 8, aquí sólo se muestra el dato.)

Esta cosecha es de después del corte del paso 8. El manifiesto dice `build 10000, oos 10, matched 10, missing_oos 9990` . El resumen del autopiloto (`20261004-154759` , paso 8) dice «OOS: 10000 → 10 por sorteo de 10»: el modo desarrollo (`criteria.yaml` , `dev`) dejó 10 en `OOS` y la cosecha, rehecha después, ve 10. `missing_oos.csv` no distingue «SQX no la escribió» de «el paso 8 la sacó». La corrida entera sin cortar es la de USDJPY B5: 10.000 / 10.000 / 10.000 emparejadas, 0 sin gemela (cosecha del 2026-10-08, recontado).

3.7 Lo que el paso 7 le entrega al paso 8

- Los `.sqx` del databank `OOS` (en disco al parar el worker) y el `Results` del Build.
- La cosecha `harvest/<P>/Results/<día>/` (métricas IS+OOS pareadas, operaciones con `sample` IS/OOS, curva diaria, `missing_oos.csv`).
- Lo que el paso 8 lee de ahí: `presencia_passed` (¿está la del build en el retest?), `sanidad_value` (nº de operaciones del OOS, de las operaciones exportadas), `estaticas_value` (el `Net profit` del OOS) y `is_trades_per_year` (`level0.py:72-80`). Las métricas del bloque OOS que faltan (sección 3.6) no las usa nadie desde la métrica.

---

4. Lo cargado: negativas y comprobaciones (no hay reglas)

El paso no tiene reglas en `criteria.yaml` (el paso 7 es de SQX, no un paso de juicio: sólo juzgan 8, 10, 12, 14 y 16). Lo que hay de validación:

qué	dónde	qué hace
regla 5: costes pactados	`sqx/projects/configure.py:170` (`pending`)	se niega a configurar si un coste está en `use: null`
nombres	regla 6, `builder`	sólo `Test_`, `Trade_`, `Research_`
`oos2`	`sqx/projects/configure.py:163-164`, `setups.py:54-55` (`setups.span`)	rechaza `oos2` a un agente autónomo; un humano puede
una tarea de OOS sin entrada	`launch/preflight.py:129-134`	«no tendría nada que probar»
dos bloques llenos	`collect.measured`	`SystemExit`: no adivina cuál mitad es el retest
identidades cambiadas	`harvest.py:96-100`, `:119-125`	las **cuenta** y las reetiqueta; no se detiene, y ni el recuento ni `alias` ni el «sólo en OOS» entran en `manifest.json` (sólo se imprimen)
falta de gemela	`level0.py:76` + `criteria.yaml:74` (R1a, `missing: fail`)	la estrategia falla en el paso 8
un rango fuera de muestra que no coincide	casi nada	`setups.py:172-175` lo escribe, ninguna comprobación lo verifica; `collect.measured` se niega si los dos bloques llevan datos, y `OOS_RANGE` no avisa si no encuentra un `<Range dateFrom dateTo>` que reescribir (`setups.py:139`)
comparación entre `Results` y `OOS`	nada	sólo la línea de `resumen.md`

---

5. Lo que no cuadra o no está medido

1. **El spread en puntos del OOS (474,91) es 1,56 veces el del Build (304,11); la ficha `costs/funded-tick-spread-vs-price` mide que en los índices el spread en puntos se mueve poco con el precio** (elasticidad de los puntos al precio: -0,64 en NIKKEI225, sobre 2017-2026; no es plana del

todo, planos son USATEC, USA500 y DJ30 con -0,14, -0,10 y -0,02; la ficha mide a Darwinex, y el spread cargado es el de FundingPips, `assets/symbols/NIKKEI225.yaml`, `spread_is.why`) **y que aplicar el porcentaje de hoy a tramos viejos subestima el coste allí 2-4 veces.** Ese texto dice a continuación que el 2026-10-04 el dueño aplicó «% a propósito: la historia paga la fricción de 2026 relativa al precio». Dos lecturas, las dos documentadas, y lo cargado es la segunda. **El efecto, con la lectura de la ficha** (FTMO JP225.cash promedió 745 pt en 2024, 487 en 2025, 920 en 2026 y 1.094 en jul-sep 2026): tanto el Build (304) como el OOS (475) pagan **menos** puntos de los medidos; el OOS por poco (queda en el mínimo de 2025) y el Build por bastante. El mismo 0,017352 % aplicado al precio de hoy (67.087,52) daría 11,64 puntos de índice = 1.164 pt SQX (calculado por el revisor: precio × 0,017352 %). El % del dueño reparte la infraestimación de forma desigual: el Build queda más barato que el OOS en proporción 1,56. Para la comparación IS-OOS, el OOS paga más que el Build y eso agranda la caída IS→OOS que ve el paso 8. Con las cifras ilustrativas del apartado 3.2, el coste extra del OOS por operación de 100 lotes es (29,98 - 19,20) + (10,49 - 6,72) = 14,55 USD; en la tabla del apartado 3.6, a 92 lotes de media, 19.2.363 paga unos 66 × 14,55 × 0,92 = 880 USD de sus 2.311 de Net profit OOS (38 %), y 13.3.390 un 14 % (calculado por el revisor; aproximado, no medido).

2. **El swap de hoy se aplica a 2020-2023** (mismo valor que al Build). La nota del fichero lo dice («es el swap de hoy; cambia con los tipos»). Cuánto pesa en la cifra del OOS: el mismo % cobra en 2020-23 en torno al 40 % de los puntos de hoy (apartado 3.2: una noche larga a 92 lotes, 13,5 USD en el Build, 21,1 en el OOS y 52,6 con el precio de hoy; calculado por el revisor). Según el signo de los tipos de la época puede sobrecargar o aliviar el OOS; no medido.
3. **El emparejamiento por nombre contra la regla del 2026-10-08.** `pairing.py:23-31` (decisión del 2026-09-26) empareja por nombre; `knowhow/sqx-format/names-reused-between-builds` (2026-10-08) dice «nunca». Hay un caso concreto en el código: el README de `pipeline/autopilot` (párrafo «One automatic retry») dice que un reintento **no vacía** el databank de salida («una repetición duplicó 1.000 a 2.000, las copias se llaman `Strategy X(1)`»; `call.keeper` lo corrige en el reintento automático). Si `OOS` guarda estrategias de otro build con el mismo nombre, `pair` las une sin error; el único síntoma es el contador «N con identidad distinta» que imprime `harvest.py:96-100` y no bloquea. Tras la limpieza de 2026-10-06 la identidad de un par legítimo no cambia: un `alias` mayor que 0 es hoy una señal fuerte de emparejamiento falso, y no se trata como tal. No he visto un caso real. Otros hechos (comprobados por el revisor en el código): `alias` y el recuento de «sólo en OOS» sólo se imprimen (`harvest.py`) y no entran en `manifest.json`, así que una corrida desatendida no deja rastro auditable; el `manifest` escribe `join: identity (sha256…), file name as fallback`, al revés de lo que hace el código; el docstring de `collect.index` dice que el nombre no puede ser la clave y `pairing.py` dice lo contrario; y `collect.index` devuelve `{identidad: fichero}`, de modo que dos `.sqx` de un mismo databank con la misma identidad (p. ej. `Strategy X` y `Strategy X(1)` de un reintento) quedan en un solo registro sin aviso. Con `alias == 0` se descarta el emparejamiento falso.
4. **`compare` y `restore` no cubren `Results` ni `OOS` en una corrida 6+7** (sección 3.5). En la ruta por defecto (`liverun`) `compare` corre tras el sync pero `restore` no se llama nunca; y una pérdida en `Results` ni siquiera la ve la regla R1a, que sólo cubre `OOS`.
5. **Descripción desfasada en dos sitios.** (a) `knowhow/export/databank-metrics-is-oos.md` explica una exportación por vista `.vw` con un worker, `sampleType 10/20/127` y esperar a `Records:`; `export_metrics.py:22-29` y `.claude/skills/export/SKILL.md` leen los `.sqx` sin arrancar SQX. (b) `.claude/skills/oos-gate/SKILL.md` dice «Harvest, with the install stopped» y «exits 1 if the worker is still alive»; el único proceso de SQX que usa la cosecha es un `orderstocsv` suelto en el

conductor sobre copias (`collect.py:96-135`). La parada del custodio sigue haciendo falta, pero para que el sync de cierre haya dejado los ficheros enteros, no porque la cosecha lo necesite. Esto último es mi lectura del código, no una medición.

6. **Los 10 códigos de `AutomaticDismissal` de la tarea OOS** (resueltos por el revisor, que descompiló con javap `listProblems()` y los `TxtShortReason*`). Son flags de bit de `com.strategyquant.tradinglib.BadStrategyException` (`SQX/internal/libs/SQTradingLib.jar`); el 128 no existe. El texto de la interfaz es «Dismiss strategies with these problems» (`SettingsRankings/views/autoDismissalPopup.html:19`). Los 10 están en `dismiss="true"` en las dos tareas (`Retest-Task1.xml:204-215` , `Build-Task3.xml:227-238`); el valor de fábrica de SQX sólo descarta el 1 (`TaskRetest/task.xml:138-147`).

código	alarma
1	sin operaciones
2	muy pocas operaciones (resultados no significativos)
4	operaciones con P/L cero
8	demasiadas operaciones cerradas en la misma barra
16	operaciones de duración cero (cerradas nada más abrir)
32	demasiadas operaciones abiertas
64	operaciones sin cerrar hasta el final del test
256	sin operaciones ejecutadas (pendientes sin llenar)
512	operación atípica (una sola enorme)
1024	demasiadas operaciones ambiguas (no se sabe si tocó SL o PT antes)

`warnings="false"` es la casilla «Add automatic checks warnings to strategy even when not used for dismissal» (`autoDismissalPopup.html:39-41`): con las 10 descartando, apagarla no cambia nada, así que `AutomaticDismissal warnings` no afecta a qué estrategias salen del `OOS` . Los datos no ven el caso de una estrategia descartada (seis cosechas con 0 sin gemela), pero los códigos 4, 16, 512 y 1024 tocan estrategias con pocas o raras operaciones y pueden dispararse justo en el OOS de 4 años. Dato del revisor (`check_b5.py` y un conteo propio, calculado por el revisor) en USDJPY B5: de 10.000 OOS emparejados, 262 tienen menos de 35 operaciones, 29 menos de 20 y el mínimo es 13; el código 2 (`dismiss="true"`) no quitó ninguna con 13, así que su umbral es bajo o relativo. No es la única vía por la que una estrategia sale del `OOS` : también el `MaxStrategies 10000` del Rankings (`Retest-Task1.xml:139`) corta un retest de más de 10.000, existen la excepción de backtest (código 10001) y el filtro de umbral (10000) (según el revisor), el sync de cierre (3.5), los duplicados `(1)` del reintento (`pipeline/autopilot/README.md`) y el colapso por identidad de `collect.index` ; sólo `DismissTooSimilarStrategies` está ausente en el OOS. Una estrategia que SQX deja fuera cuenta como fallo de la R1a (`level0.py` , `criteria.yaml:74`); el dossier no puede decir por qué, porque `missing_oos.csv` (`harvest.py`) sólo guarda `identity` y `strategy_build` . 7. **La métrica de OOS no trae nº de operaciones, drawdown ni Max DD %.** Cualquier regla que quiera la rentabilidad sobre el drawdown de OOS debe sacarlo de las operaciones o de `Ret/DD` (sí está) y no existe hoy un hecho de «drawdown en OOS» en esta tabla. 8. **Primera operación del OOS entre 7 y 35 días después del inicio.** No hay evidencia de que el calentamiento de indicadores se coma ese principio: el cociente mediano entre ese retraso y la brecha media entre operaciones es 1,06 (calculado por el revisor; apartado 3.1), el tiempo esperado hasta la primera señal. Ninguna barra anterior al 2020-01-01 está cargada

en el OOS; si el calentamiento existe, le quita operaciones al OOS. Los bordes de EETUS (apartado 3.6) siguen sin comprobar, pero el `Setup dateTo` limita las operaciones. 9. **`R Expectancy` redondeada a dos decimales por la vista.** La columna de métricas que sale de `sqxview.py` no es la de la vista; la ficha dice que las demás coinciden hasta 6e-8. No lo he reverificado. 10. **Un desfase de comentario en `assets/_build.yaml`:** en el árbol de trabajo hay una casilla `precision: 1 # un minuto — lento y fino` (línea 249, de los escenarios del MC Retest, no del paso 7; `testPrecision` 2 = un minuto está en la línea 87, y la `precision` del MC, líneas 204-205, es otra cosa). El valor es 1 y el comentario dice un minuto. No afecta al paso 7, pero confunde al leer la precisión. 11. **El IS parece empezar tarde.** En el IS de la corrida de Nikkei las 10 primeras operaciones llegan 148-184 días después del 2011-09-19 (una a 627); el cociente mediano frente a la brecha media entre operaciones es 9,0, frente a 1,06 en el OOS (calculado por el revisor, desde las operaciones de la cosecha). Eso sí parece un arranque comido, por feed o por calentamiento. No se ha mirado `NIKKEI225_M1` real desde 2011-09-19: hasta hacerlo, el IS no está dado por entero. 12. **Dos cosechas del mismo día del mismo conjunto de estrategias se sobrescriben.** `set_aside` sólo aparta la del mismo día si es de otro conjunto; con el mismo, la segunda pasada (`harvest_dir(..., date.today())`) borra la primera y sus cifras no se guardan. 13. **El `StopCondition` de 10 minutos de la tarea `OOS`** (`Retest-Task1.xml:216`, igual que el del Build, `Build-Task3.xml:239`): `buildcap.py` sólo hace cumplir el tope del Build y no se ha comprobado si SQX lo aplica a un retest. Las seis cosechas completas sugieren que no corta, pero no está escrito.

---

6. Cierre: decisiones que le quedan al dueño

1. **¿Qué spread paga el OOS?** Hoy, 0,0174 % del precio medio del tramo (474,91 puntos). Si prefieres la lectura de la ficha (puntos casi constantes), el OOS pagaría un spread en puntos cercano al del Build, y los dos por encima de los 304 y 475 puntos de hoy (487-1.094 pt medidos en FTMO JP225.cash). *Gana:* coherencia con el comportamiento medido de los índices. *Cuesta:* un OOS más barato que el actual, un Build más caro y la reescritura de `assets/symbols/<activo>.yaml` en todos los índices; un cambio de coste invalida las cifras ya cosechadas.
2. **¿Swap histórico o de hoy?** Hoy es el de hoy en todos los tramos. *Gana (histórico):* un OOS que no cobra tipos que no existían. *Cuesta:* no hay tabla de swap por años en `assets/`; habría que construirla.
3. **¿El emparejamiento por nombre se endurece?** Bastaría tratar `alias > 0` como una parada en `harvest.py` (hoy sólo lo cuenta, y ni siquiera lo guarda en `manifest.json`). *Gana:* nadie puede unir dos estrategias distintas. *Cuesta:* parar la cosecha en un caso que hoy sigue; puede interrumpir un arranque por una falsa alarma.
4. **¿Se vigilan `Results` y `OOS` en una corrida 6+7?** Hoy sólo se ve la línea `OOS: 0 → N` del resumen (una pérdida en `OOS` la ve la R1a; una en `Results`, nadie). Una comprobación «`OOS` tiene tantas como `Results`» sería barata (los dos recuentos ya están en `moved`). *Gana:* una pérdida por sync deja de ser silenciosa. *Cuesta:* decidir qué hacer cuando no cuadra (parar, reintentar, o seguir con la presencia en R1a).
5. **¿Qué alarmas de `AutomaticDismissal` descartan en el OOS?** Hoy las 10 (tabla del apartado 5.6), y es una de las vías por las que SQX puede dejar una estrategia fuera del `OOS`. *Gana:* saber que un retest «limpio» puede perder estrategias legítimas (por ejemplo, con pocas operaciones: códigos 2, 4, 16, 512 y 1024). *Cuesta:* probarlo con una estrategia de juguete en el conductor para ver cuál dispara.
6. **¿Se actualizan las dos descripciones desfasadas** (`databank-metrics-is-oos.md` y `oos-gate/SKILL.md`)? Es trabajo del documentador, no de este dossier.